Claude Code hackathon guide

Oct 17, 2026 · @William

Read the workflow section first; it is the plan. The AI-generated reference at the bottom covers Windows setup and extra tips.

Critical: the workflow (read this first)

The idea matters most. Spend your time on the concept before touching code. This video was the first result for "Hackathon Claude Code workflow". The creator builds physical products, but the workflow still applies. Watch the whole thing.

1. **PRD: Opus 5.5 High/xHigh for everything.** Write a few paragraphs on what the app is (a few hundred words minimum; dictation like Willow Voice, which is free, makes this fast). Expand it with grill-me, which interviews you. Review the app flow with lavish-axi, then loop between the two (untested, so experiment).
2. **UI mocks.** Once the PRD is final, have Claude Code generate mocks (sample tutorial). Pre-install the UI skills from this video.
3. **Build.** Split the work into sub-tasks first (Claude can do it), build with Haiku 5.5 (high) subagents, and have Sonnet 5.5 (high) subagents review every change. Nobody reviews code by hand, so agent review is the safety net.
4. **Test.** Use agent-browser so agents test every part of the site and debug continuously.

Context and teamwork

- Compact at about 60% context; for Haiku, at 35%.
- Parallelizing is hard: rotate usage limits or build separate parts of the app. Do the design phase together as a team.

NOTE: You can also just try the dude's plugin: https://github.com/23jmo/jmo-skills.

AI-generated reference

Written by Claude from web searches on Oct 17, 2026. Skim it; the workflow above is the plan.

Windows setup

1. Install Git for Windows. It is recommended so Claude Code can use Git Bash; otherwise it falls back to PowerShell.

2. In PowerShell, run `irm https://claude.ai/install.ps1 | iex`, then run `claude` in your project folder (needs Windows 10 1809 or newer).
3. Install Node.js LTS, which `npx skills`, `agent-browser` and your app need.
4. Set your git identity (`git config --global user.name` and `user.email`) and sign in with `gh auth login`.
5. Use native Windows or WSL, not both; each keeps its own config and login.
6. If commands time out, exclude `.git/` and `node_modules/` from Windows Defender.

Skills and git

- Install skills with `npx skills add <owner/repo> -s <skill> -g`, for example `npx skills add mattpocock/skills -s grill-me -g`.
- agent-browser also needs its CLI: `npm install -g agent-browser`, then `agent-browser install`. An older third-party note reports issues in PowerShell and CMD; if it flakes, run it from WSL.
- No git skill is needed. Put these rules in CLAUDE.md: commit after each working sub-task, one short branch per person, never force-push to main.

Habits

- Run `/init` to create CLAUDE.md (stack, run and test commands, conventions) and update it after every mistake.
- Press Shift+Tab for plan mode before big builds; use `/clear` between unrelated tasks.
- Deploy a hello-world early, keep `main` working, cut scope, and rehearse the demo.

Subagent orchestration

- Run Opus as the orchestrator with two custom agents in `.claude/agents/`: `builder` (`model: haiku`) and `reviewer` (`model: sonnet`). `/agents` can generate them; write descriptions that say when to use them.
- Parallelize only across different files (frontend, backend, database). Subagents can't spawn other subagents.
- Give each subagent file paths, the goal and success criteria; it starts fresh and returns only its final message.
- Use the built-in Haiku-powered Explore subagent for codebase search.
- For isolation, use worktrees (`claude -w`, or tell Claude to "use worktrees for your agents"). Worktrees share neither `.env` nor `node_modules`, so copy the env file and run install in each. Merge branches one at a time.
- Three to five parallel agents is the commonly cited sweet spot. With no human code review, lean on the reviewer agent and agent-browser tests.

WSL vs Windows

- **Mixed team (Mac + Windows):** Use WSL2. Claude Code goes inside the WSL terminal, and the repo lives in the WSL home folder, not under `/mnt/c`. You then run the same bash-style shell, paths and scripts as your Mac teammates, which avoids most "works on my machine" bugs. It's also closer to Linux hosting. agent-browser has also been reported flaky on native Windows.
- **All Windows:** Native with Git for Windows is fine and has fewer moving parts. Switch to WSL only if a tool breaks.
- **Either way:** Add a `.gitattributes` with `* text=auto eol=lf` so Windows doesn't commit CRLF line endings. Also commit your lockfile, pin the Node version (`.nvmrc`), and keep a `.env.example` so everyone's setup matches.

Hosting (depends on your stack, which I don't know yet)

- **Frontend or full-stack JS (e.g. Next.js):** Vercel is the easiest. Connect the GitHub repo and every push to `main` redeploys, with preview URLs per branch. Netlify works similarly.
- **Separate backend or API:** Render, Railway or Fly.
- **Database and auth:** Supabase or Firebase.
- **Habits:** Deploy a hello-world in the first hour. One person owns the hosting account and invites the rest. Keep secrets in the host's environment variable settings, never in git. Free-tier limits change, so check them before the event.

Sources

- Claude Code setup
- Install Claude Code on Windows
- Claude Code Subagents (2026)
- Sub-agent parallel vs sequential patterns
- Git Worktrees + Claude Code
- Boris Cherny setup summary
- npx skills reference
- agent-browser installation
- agent-browser MCP Windows note